COMPUTER ORGANISATION & OPERATING SYSTEMS / CO1 / 8 MARKS

System Detective

CIE-1 machine inspection and memory-locality experiment

Roll number: _____ **Name:** _____

Machine: Omarchy Linux, x86_64 **System commands:** 27 September 2026 **Locality/screenshots:** 27-28 September 2026

Host findings

The machine inspection was run directly on the Linux host, not in a simulated environment. The CPU report identifies an AMD Ryzen 7 7435HS with 8 cores and 16 logical CPUs. The reported instruction flags include SSE2, AVX, AVX2, and AES. `free -h` reported 15 GiB total memory, 6.2 GiB used, and 9.2 GiB available at the command-capture time.

Architecture x86_64
L3 cache 16 MiB
Logical CPUs 16 (8 cores x 2 threads)
Matrix 4096 x 4096 uint64_t / 128 MiB

Locality experiment

The same initialized matrix was read once in each order. In C row-major layout, adjacent columns are contiguous. The column-major loop jumps 32 KiB between consecutive reads of a column. The program was compiled with GCC using `-O2 -fno-tree-vectorize`, disabling automatic loop vectorization for both orders. Both orders produced the same checksum. Across three paired runs, the median row-major time was 0.009116 s and the median column-major time was 0.823384 s, about 90.3x slower for the strided traversal on this run.

RUN	ORDER	SECONDS	CHECKSUM
1	row-major	0.009115695	2097144125
1	column-major	0.824955201	2097144125
2	column-major	0.823384134	2097144125
2	row-major	0.008714712	2097144125
3	row-major	0.010915033	2097144125
3	column-major	0.804405443	2097144125

> Benchmark timings are specific to this machine and run conditions. CPU load, compiler, kernel, and cache state can change measured times.

LIVE-DATA DASHBOARD

One view, current host

The dashboard refreshes CPU identification, memory availability, and the five highest `ps %CPU` rows every three seconds. Process states are displayed as reported by `ps`; the locality timings are read from the saved benchmark results and can be refreshed with the run button.

Figure 1. Dashboard locality panel showing the host's measured medians and paired-run count.

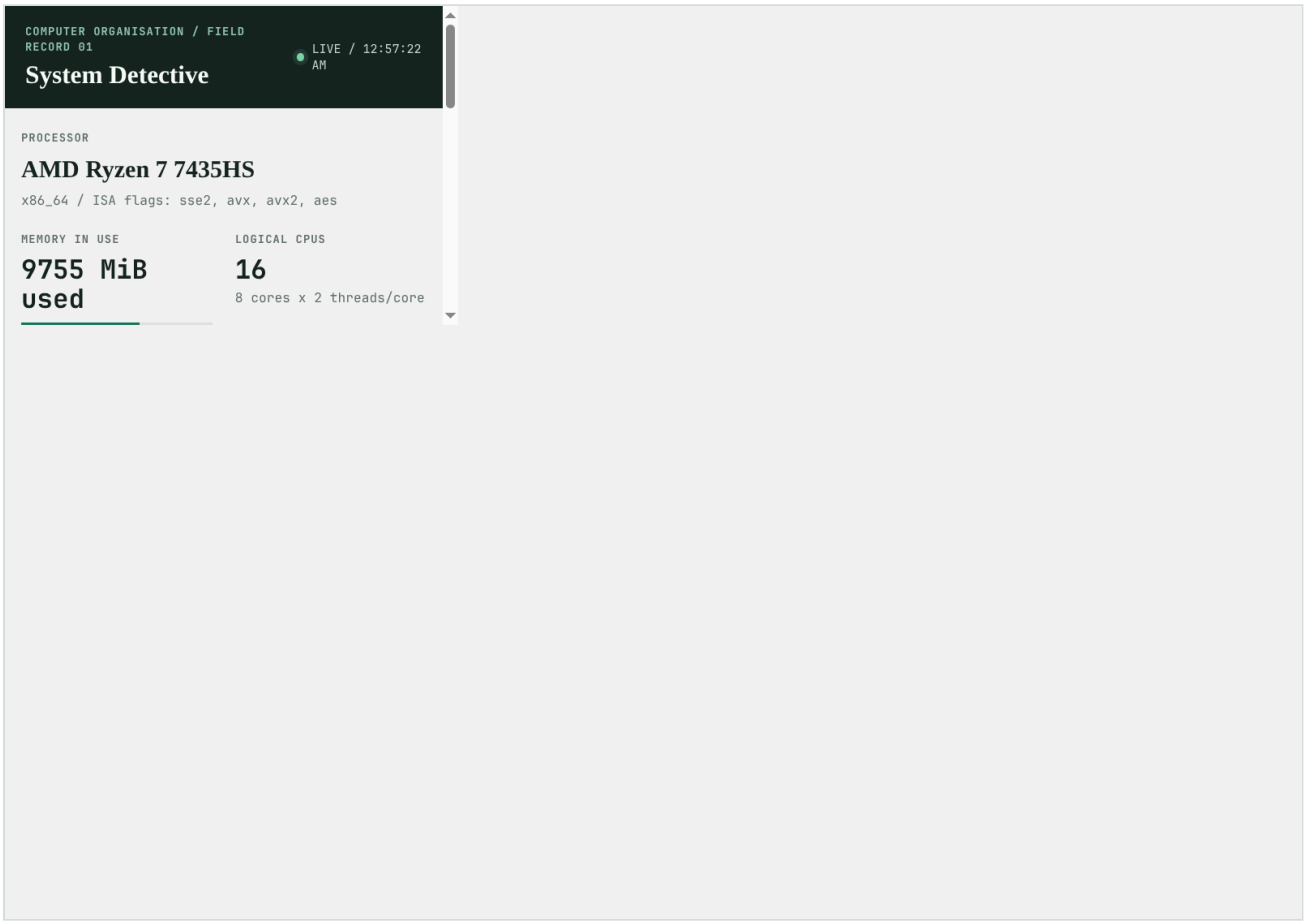


Figure 2. Dashboard running locally at 127.0.0.1:8765. Values visible in this capture are live readings at screenshot time.

> **Commands captured:** `lscpu`, `free -h`, `top -b -n 1`, `ps -eLf`, and `strace -c /usr/bin/true`. Full stdout/stderr is retained in the `evidence/` directory alongside the screenshots.

To reproduce the dashboard: compile `locality.c` with GCC, then run `/usr/bin/python dashboard/server.py` and open the local URL. No external services or hardcoded host readings are needed.

RAW COMMAND EVIDENCE

Five required Linux tools



Figure 3. `lscpu` output: architecture, CPU model, and topology.



Figure 4. `free -h` output: memory and swap at capture time.

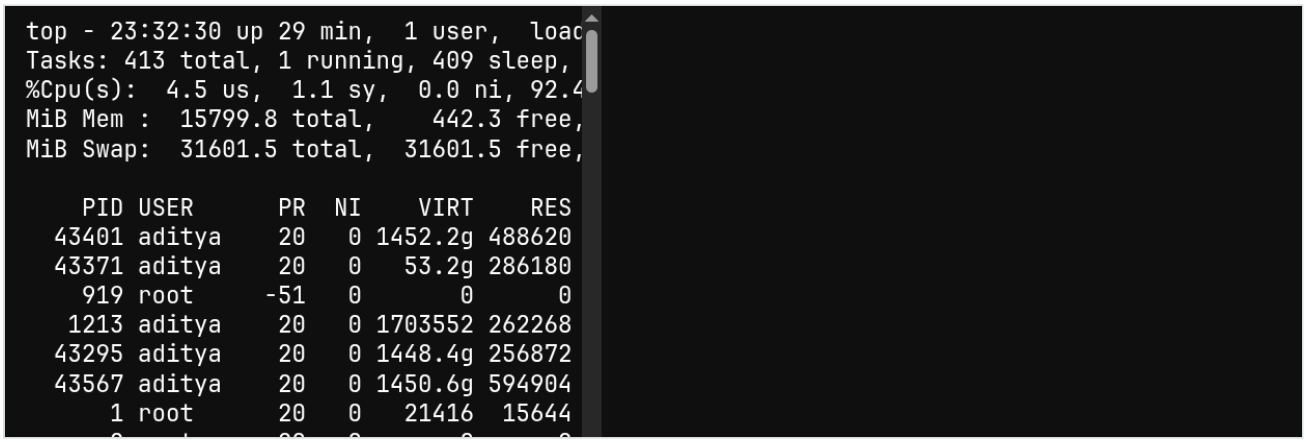


Figure 5. `top -b -n 1` batch snapshot.

```
UID          PID    PPID     LWP  C NLWP
root           1       0       1  0    1
root           2       0       2  0    1
root           3       2       3  0    1
root           4       2       4  0    1
root           5       2       5  0    1
root           6       2       6  0    1
root           7       2       7  0    1
root           8       2       8  0    1
root           9       2       9  0    1
root          11       2      11  0    1
root          14       2      14  0    1
root          15       2      15  0    1
root          16       2      16  0    1
root          17       2      17  0    1
root          18       2      18  0    1
```

Figure 6. `ps -eLf` process and thread listing.

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         1           read
  0.00    0.000000           0         2           close
  0.00    0.000000           0         2           fstat
  0.00    0.000000           0         8           mmap
  0.00    0.000000           0         3           mprotect
  0.00    0.000000           0         1           brk
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         1           arch_prctl
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         2           openat
  0.00    0.000000           0         2           newfstatat
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         2           getrandom
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        30         1 total
```

Figure 7. `strace -c /usr/bin/true` syscall summary.

> The screenshot readings are snapshots, not constants. Rerunning the saved commands on this host produces fresh values.